TEMA 3

Diseño y realización de pruebas

Estrategias, casos de prueba, depuración y automatización con JUnit 5

Propósito del tema

Comprender el proceso, utilizar las herramientas con criterio y documentar decisiones reproducibles.

ENTORNOS DE DESARROLLO · 1.º DAM

Qué aprenderás

- Comprender la finalidad y los principios de las pruebas de software.
- Diseñar casos mediante técnicas de caja negra y caja blanca.
- Documentar ejecución, resultados, incidencias y causas.
- Automatizar pruebas unitarias en Java con JUnit 5.

Ruta de aprendizaje

Conceptos → decisiones técnicas → ejemplos en Java → comprobación y documentación.

Índice del tema

1. Por qué probamos software
2. Principios y estrategia
3. Anatomía de un caso de prueba
4. Niveles de prueba
5. Tipos de prueba del sistema
6. Caja negra: particiones de equivalencia
7. Caja negra: valores límite y conjetura
8. Caja blanca y cobertura
9. Complejidad ciclomática y camino básico
10. Pruebas de bucles y flujo de datos
11. Plan y documentación de pruebas
12. Incidencias y análisis causal
13. Depuración y regresión
14. Pruebas unitarias con JUnit 5
15. Aserciones, excepciones y parametrización
16. Automatización, cobertura y optimización

1. Por qué probamos software

Probar software consiste en ejecutarlo o revisarlo de forma planificada para obtener información sobre su calidad y sus riesgos. El objetivo no es demostrar que nunca fallará, sino descubrir defectos y aumentar la confianza antes de la entrega.

Mentalidad de prueba

Una prueba eficaz intenta encontrar situaciones en las que el producto no cumple lo esperado. Encontrar un defecto es información valiosa, no un fracaso personal.

Término	Significado
Error humano	Acción o decisión incorrecta durante el trabajo
Defecto	Problema introducido en un requisito, diseño, código o dato
Fallo	Comportamiento observable incorrecto al ejecutar
Incidencia	Registro de un resultado que necesita investigación

2. Principios y estrategia

- Las pruebas muestran presencia de defectos, no su ausencia absoluta.
- Probarlo todo suele ser imposible: se prioriza por riesgo y valor.
- Diseñar pruebas pronto reduce el coste de corregir.
- Los defectos se concentran con frecuencia en determinadas áreas.
- Repetir siempre las mismas pruebas reduce su capacidad de descubrimiento.
- La estrategia depende del contexto: criticidad, usuarios, tecnología y cambios.

Independencia

Una mirada distinta reduce sesgos. Puede ser otra persona, una revisión, una herramienta o un caso derivado directamente del requisito.

3. Anatomía de un caso de prueba

Un caso de prueba especifica condiciones previas, datos de entrada, pasos, resultado esperado y criterio de superación. Debe poder repetirse y relacionarse con un requisito o riesgo.

Campo	Ejemplo
Identificador	CP-LOGIN-003
Objetivo	Rechazar una contraseña incorrecta
Precondición	Usuario activo registrado

Campo	Ejemplo
Entrada	Correo válido y contraseña errónea
Resultado esperado	Acceso denegado sin revelar qué dato falló
Resultado real	Se completa durante la ejecución

Oráculo de prueba

El resultado esperado debe definirse antes de ejecutar. Comparar solo con lo que el programa hace puede normalizar un defecto.

4. Niveles de prueba

Nivel	Qué comprueba	Defectos habituales
Unidad	Método o clase de forma aislada	Cálculos, condiciones, excepciones
Integración	Colaboración entre componentes	Contratos, formatos, orden y comunicación
Sistema	Producto completo en entorno representativo	Requisitos, rendimiento, seguridad y compatibilidad
Aceptación	Adecuación para el uso y necesidades del cliente	Flujos de negocio y criterios de aceptación

```
unidad → integración → sistema → aceptación
  más local                        más próximo al uso real
```

Los niveles no sustituyen unos a otros. Una prueba de sistema puede revelar un fallo, mientras que las pruebas unitarias ayudan a localizarlo con rapidez.

5. Tipos de prueba del sistema

Tipo	Pregunta
Funcional	¿Realiza las funciones y reglas especificadas?
Rendimiento	¿Responde dentro de tiempos y consumo aceptables?
Carga y estrés	¿Cómo se comporta en condiciones normales y extremas?
Seguridad	¿Protege datos y resiste accesos no autorizados?
Recuperación	¿Restablece el servicio tras un fallo?
Compatibilidad	¿Funciona en plataformas y configuraciones previstas?
Usabilidad y accesibilidad	¿Puede utilizarse con eficacia por las personas destinatarias?

6. Caja negra: particiones de equivalencia

Las técnicas de caja negra diseñan pruebas desde requisitos, entradas y salidas sin considerar la implementación interna. Las particiones de equivalencia agrupan valores que deberían producir un comportamiento similar.

Si una edad válida está entre 18 y 65, pueden definirse tres clases: menor de 18, entre 18 y 65, y mayor de 65. Se selecciona al menos un representante de cada clase.

Clase	Representante	Esperado
edad < 18	17	Rechazo
18 ≤ edad ≤ 65	40	Aceptación
edad > 65	66	Rechazo
entrada no numérica	"veinte"	Error controlado

7. Caja negra: valores límite y conjetura

Los defectos aparecen con frecuencia en los límites. Para el intervalo 18..65 se prueban 17, 18, 19, 64, 65 y 66. En listas se revisan la primera y última posición, la colección vacía y el tamaño máximo.

Técnica	Cómo genera casos
Valores límite	Selecciona justo antes, en y justo después del borde
Tabla de decisión	Combina condiciones y acciones de reglas de negocio
Transición de estados	Prueba eventos válidos e inválidos según el estado
Conjetura de errores	Usa experiencia: cero, vacío, null, duplicados, formatos extraños

Eficiencia

Las técnicas permiten elegir pocos casos con alto poder de detección. No se trata de acumular datos al azar.

8. Caja blanca y cobertura

Las técnicas de caja blanca utilizan la estructura interna para recorrer sentencias, decisiones, condiciones, bucles y flujos de datos. Complementan, pero no sustituyen, las pruebas derivadas de requisitos.

Cobertura	Qué exige
Sentencias	Ejecutar cada instrucción al menos una vez
Ramas	Ejecutar resultados verdadero y falso de cada decisión
Condiciones	Hacer verdadera y falsa cada condición relevante
Caminos	Recorrer combinaciones de flujo; puede ser inabarcable

> **Cobertura no equivale a calidad**
>
> Un 100 % de líneas ejecutadas no garantiza buenas aserciones, requisitos completos ni ausencia de defectos.

9. Complejidad ciclomática y camino básico

La complejidad ciclomática estima el número de caminos linealmente independientes del grafo de control. Para un componente conectado puede calcularse como V(G) = E − N + 2 o, de forma práctica, número de decisiones + 1.

```
public boolean puedeAcceder(int edad, boolean autorizado) {
    if (edad >= 18 && autorizado) {
        return true;
    }
    return false;
}
// Decisión principal + 1 → al menos dos caminos básicos
```

Una complejidad alta indica más combinaciones que entender y probar. Es una señal para revisar diseño, no una sentencia automática sobre la calidad.

10. Pruebas de bucles y flujo de datos

Bucles

Para un bucle simple se consideran cero iteraciones, una, dos, un valor típico, el máximo previsto y, si procede, justo por encima. En bucles anidados se controla primero el interior y se limita la explosión de combinaciones.

Flujo de datos

Se estudian definiciones y usos de variables. Una cadena definición-uso es un camino desde una asignación hasta una lectura sin otra redefinición intermedia. Ayuda a detectar valores sin inicializar, asignaciones inútiles o usos incorrectos.

Caso mínimo

Una colección vacía es un caso de primera clase: puede descubrir divisiones por cero, accesos fuera de rango y acumuladores mal inicializados.

11. Plan y documentación de pruebas

La documentación permite coordinar, repetir y auditar las pruebas. Debe ser suficiente para saber qué se probó, con qué versión y entorno, qué ocurrió y qué queda pendiente.

Documento	Contenido
Plan de pruebas	Alcance, estrategia, recursos, calendario, riesgos y criterios de cierre
Diseño de pruebas	Técnicas, condiciones, cobertura y trazabilidad
Caso o procedimiento	Datos, pasos, esperado y precondiciones
Histórico	Ejecuciones, entorno, versión, responsable y resultados
Informe de incidencias	Síntoma, pasos, evidencia, severidad y estado
Resumen	Cobertura, defectos, tendencias y recomendación

12. Incidencias y análisis causal

Una incidencia útil permite reproducir el fallo. Debe describir versión, entorno, precondiciones, pasos, resultado esperado, resultado real y evidencia. La severidad mide impacto; la prioridad decide cuándo atenderla.

- ¿Qué ocurrió y dónde se observó?
- ¿Qué condición permitió que el defecto se introdujera?
- ¿Por qué no fue detectado antes?
- ¿Qué corrección elimina la causa y no solo el síntoma?
- ¿Qué cambio de proceso o prueba puede prevenir la repetición?

Finalidad

El análisis causal busca aprendizaje del sistema de trabajo; no asignar culpa a una persona.

13. Depuración y regresión

Una prueba revela un fallo; la depuración localiza su causa y permite corregirla. Tras el cambio se repite el caso que fallaba y un conjunto de pruebas de regresión para comprobar que no se ha roto comportamiento existente.

Herramienta	Uso
Punto de ruptura	Detenerse antes del comportamiento sospechoso
Pila de llamadas	Reconstruir la cadena de métodos
Variables	Comparar el estado real con el esperado
Evaluar expresión	Comprobar una hipótesis sin modificar el programa
Step Over / Into / Return	Controlar el avance de la ejecución

Protocolo tras una corrección

- Añadir o conservar un caso automatizado que reproduzca el defecto.
- Aplicar el cambio mínimo que elimine la causa identificada.
- Ejecutar de nuevo el caso fallido y las pruebas relacionadas.
- Revisar efectos laterales mediante la suite de regresión.
- Actualizar incidencia, documentación y versión del producto.

> **Defecto que no vuelve**
>
> Una corrección queda realmente cerrada cuando existe evidencia repetible de que funciona y una prueba protege frente a su reaparición.

14. Pruebas unitarias con JUnit 5

JUnit 5 permite expresar pruebas como código ejecutable. Cada prueba debe ser pequeña, independiente, determinista, rápida y legible. Una estructura frecuente es Preparar → Actuar → Comprobar.

```
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

class CalculadoraTest {
    @Test
    void sumaDosEnteros() {
        // Preparar y actuar
        int resultado = Calculadora.sumar(2, 3);
        // Comprobar
        assertEquals(5, resultado);
    }
}
```

Anotación	Finalidad
@Test	Marca un método como prueba
@BeforeEach	Prepara un estado nuevo antes de cada prueba
@AfterEach	Libera recursos después de cada prueba
@BeforeAll / @AfterAll	Preparación o cierre compartido de la clase
@DisplayName	Describe el comportamiento en lenguaje legible

15. Aserciones, excepciones y parametrización

Aserción	Comprueba
assertEquals	Igualdad entre esperado y real
assertTrue / assertFalse	Condición booleana
assertNull / assertNotNull	Ausencia o presencia de referencia
assertArrayEquals	Contenido de arrays
assertThrows	Tipo de excepción esperada

```
@Test
void importeNegativoLanzaExcepcion() {
    Cuenta cuenta = new Cuenta(100);
    assertThrows(IllegalArgumentException.class,
        () -> cuenta.ingresar(-10));
}
```

```
@ParameterizedTest
@ValueSource(ints = {17, 18, 65, 66})
void compruebaLimitesDeEdad(int edad) {
    assertEquals(edad >= 18 && edad <= 65, esValida(edad));
}
```

Pruebas parametrizadas

Ejecutan la misma lógica con varios datos y hacen visibles las clases de equivalencia y los valores límite.

16. Automatización, cobertura y optimización

Las pruebas automatizadas pueden ejecutarse en cada cambio y en integración continua. La cobertura ayuda a localizar código no ejercitado, pero debe interpretarse junto con la calidad de los casos y los riesgos del producto.

La optimización debe comenzar con mediciones. Se perfila CPU, memoria o entrada/salida, se identifica el cuello de botella, se realiza un cambio pequeño y se vuelven a ejecutar pruebas funcionales, de regresión y rendimiento.

```
medir → localizar → formular hipótesis → optimizar
   ↑                                        ↓
   └──── comparar + pruebas de regresión ──┘
```

- Evitar optimizaciones prematuras basadas solo en intuición.
- Conservar una línea base para comparar antes y después.
- Controlar complejidad algorítmica, asignaciones, consultas y operaciones de E/S.
- No aceptar una mejora de velocidad que cambie resultados o empeore mantenibilidad sin justificación.

Contenido traducido

Los términos técnicos de optimización y cobertura se han normalizado al español, manteniendo entre paréntesis el término inglés cuando es útil.

Síntesis y autoevaluación

- Las pruebas aportan información sobre defectos, riesgos y confianza.
- Caja negra deriva casos del comportamiento; caja blanca usa la estructura interna.
- Un caso reproducible necesita entradas, precondiciones y resultado esperado.
- JUnit 5 convierte las comprobaciones unitarias en una red de regresión automática.
- La cobertura y el rendimiento deben medirse e interpretarse, no perseguirse como cifras aisladas.

Reto final · Comprueba que dominas el tema

Diseña una estrategia para una clase Cuenta: niveles de prueba, particiones y límites, casos de excepciones, cuatro pruebas JUnit 5, cobertura esperada y formato de informe de incidencia.

Material base

Tema reorganizado, ampliado y redactado en español a partir de entornos-diseño-y-realización-de-pruebas.pdf, aportado por el docente. No se reproducen los ejercicios del documento original.